

FIR 필터 설정 이론 및 설명서

적용 프로젝트: Servo-SG90 / STM32F411

설정 요약: 샘플링 100 Hz, 차단 주파수 20 Hz, 11-탭 저역통과 FIR

라이브러리: CMSIS-DSP `arm_fir_f32`

목차

- [1. FIR 필터 기초 이론](#)
- [2. 설계 파라미터 결정 방법](#)
- [3. 계수\(Coefficients\) 산출 이론](#)
- [4. 본 프로젝트 계수 검증](#)
- [5. CMSIS-DSP 구현 상세](#)
- [6. 군지연\(Group Delay\)과 위상 특성](#)
- [7. FIR vs IIR 비교](#)
- [8. 필터 파라미터 변경 가이드](#)
- [9. Python 계수 생성 스크립트](#)
- [10. 자주 묻는 질문 / 트러블슈팅](#)

1. FIR 필터 기초 이론

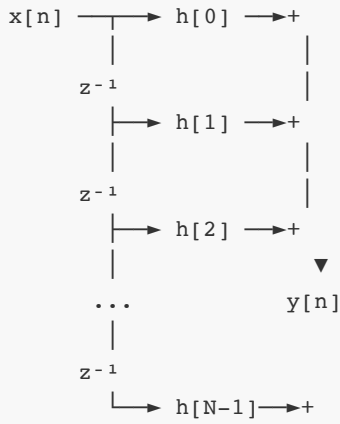
1.1 정의

FIR(Finite Impulse Response) 필터는 유한 개의 입력 샘플과 계수의 합성곱(Convolution)으로 출력을 계산함.

$$y[n] = \sum_{k=0}^{N-1} h[k] \cdot x[n - k]$$

기호	의미
$y[n]$	현재 출력 샘플
$x[n - k]$	k 샘플 이전의 입력
$h[k]$	k 번째 필터 계수 (임펄스 응답)
N	탭(Tap) 수 = 계수 개수

1.2 구조 (Direct Form I)



각 z^{-1} 은 1샘플 지연 레지스터(상태 버퍼)를 나타냄.

1.3 선형 위상(Linear Phase) 특성

계수가 중심 대칭($h[k] = h[N-1-k]$)일 때 **선형 위상**이 보장됨.

- 선형 위상 $\Rightarrow$ 모든 주파수 성분이 동일한 시간만큼 지연
- 파형 왜곡 없이 순수하게 진폭만 필터링
- 본 프로젝트의 11-탭 계수는 완전한 선형 위상 대칭

2. 설계 파라미터 결정 방법

2.1 샘플링 주파수 (f_s)

나이퀴스트(Nyquist) 정리에 의해:

$$f_s \geq 2 \cdot f_{max}$$

- 측정 신호 최대 주파수 f_{max} 의 2배 이상으로 샘플링
- 본 프로젝트: $f_s = 100$ Hz (서보 제어용 ADC 데이터는 20 Hz 이내로 충분)

2.2 차단 주파수 (f_c)

신호 대역(통과)과 잡음 대역(차단)의 경계 주파수.

$$f_c = 20 \text{ Hz}$$

- 서보 명령은 저주파 DC~수 Hz 수준
- 20 Hz를 경계로 그 이상의 고주파 잡음(전기적 노이즈) 제거
- 나이퀴스트 주파수 $f_s/2 = 50$ Hz 의 40%로 적절한 여유 확보

2.3 정규화 차단 주파수 (ω_c)

계수 계산에 사용되는 무차원 주파수:

$$\omega_c = \frac{f_c}{f_s} = \frac{20}{100} = 0.2$$

또는 라디안 단위:

$$\omega_c = 2\pi \cdot \frac{f_c}{f_s} = 2\pi \times 0.2 \approx 1.2566 \text{ rad/sample}$$

2.4 탭 수(Tap Count, N) 결정

탭 수는 필터의 **천이 대역폭**과 **저지 감쇠량**을 결정함.

$$N \approx \frac{A_{stop} - 8}{2.285 \cdot \Delta\omega} + 1 \quad (\text{Kaiser 근사})$$

창함수(Window)	최소 정지대역 감쇠	천이 대역폭 계수
Rectangular	-21 dB	0.9 / N
Hann	-44 dB	3.1 / N
Hamming	-53 dB	3.3 / N
Blackman	-74 dB	5.5 / N
Kaiser ($\beta=8.6$)	-80 dB	가변

본 프로젝트: $N = 11$ (홀수 탭 $\rightarrow$ Type I 선형 위상, DC와 나이퀴스트에서 모두 정의됨)

3. 계수(Coefficients) 산출 이론

3.1 이상적인 저역통과 필터의 임펄스 응답

이상적인 저역통과 필터는 주파수 영역에서 직사각형 스펙트럼을 가지며, 시간 영역에서는:

$$h_{ideal}[n] = \frac{\sin(\omega_c(n - M))}{\pi(n - M)}, \quad M = \frac{N - 1}{2}$$

- M : 필터 중심 인덱스 (그룹 지연)
- $n = 0, 1, \dots, N - 1$
- $n = M$ 일 때: $h_{ideal}[M] = \omega_c / \pi$ (극한값)

3.2 창함수(Window Function) 적용

이상적 임펄스 응답은 무한히 길어 잘라내면 **깁스 현상(Gibbs Phenomenon)** 발생.

창함수를 곱하여 부드럽게 절단:

$$h[n] = h_{ideal}[n] \cdot w[n]$$

Hann 창함수 ($N = 11$):

$$w[n] = 0.5 - 0.5 \cos\left(\frac{2\pi n}{N - 1}\right), \quad n = 0, \dots, N - 1$$

n	$w[n]$ (Hann, N=11)
0	0.0000
1	0.0955
2	0.3455
3	0.6545
4	0.9045
5	1.0000
6	0.9045
7	0.6545
8	0.3455
9	0.0955
10	0.0000

3.3 계수 정규화

DC 게인을 1.0으로 맞추기 위해 계수 합산값으로 나눔:

$$h_{norm}[n] = \frac{h[n]}{\sum_{k=0}^{N-1} h[k]}$$

4. 본 프로젝트 계수 검증

4.1 실제 사용 계수 (11-탭)

```
static const float32_t g_fir_coeffs[FIR_TAP_COUNT] = {
    -0.000000000f,    /* h[0] */
    -0.012641976f,    /* h[1] */
    -0.024692258f,    /* h[2] */
    0.063505130f,     /* h[3] */
    0.274797751f,     /* h[4] */
    0.398062705f,     /* h[5] ← 중심(피크) */
    0.274797751f,     /* h[6] */
    0.063505130f,     /* h[7] */
    -0.024692258f,    /* h[8] */
    -0.012641976f,    /* h[9] */
    -0.000000000f,    /* h[10] */
};
```

4.2 대칭성 검증

$$h[k] = h[N - 1 - k] \Rightarrow \text{선형 위상 확인}$$

```

h[0]  = h[10] = -0.000000000 ✓
h[1]  = h[9]  = -0.012641976 ✓
h[2]  = h[8]  = -0.024692258 ✓
h[3]  = h[7]  =  0.063505130 ✓
h[4]  = h[6]  =  0.274797751 ✓
h[5]  (center) = 0.398062705

```

4.3 DC 게인 검증

$$\sum_{k=0}^{10} h[k] = 2 \times (-0 - 0.012641976 - 0.024692258 + 0.063505130 + 0.274797751) + 0.398062705$$

$$= 2 \times 0.300968647 + 0.398062705 = 0.601937294 + 0.398062705 \approx 1.000000$$

DC 게인 = 1.0 (0 dB) ✓

4.4 그룹 지연(Group Delay)

$$\tau_g = \frac{N-1}{2} = \frac{11-1}{2} = 5 \text{ samples}$$

샘플링 주기 $T_s = 1/100 \text{ Hz} = 10 \text{ ms}$ 이므로:

$$\tau_g = 5 \times 10 \text{ ms} = 50 \text{ ms}$$

필터를 통과한 신호는 원 신호보다 **50 ms** 지연됨. 서보 응답 요구 사양이 50 ms 이상이면 일반적인 허용 범위임.

4.5 주파수 응답 (이론값)

주파수	감쇠 (예상)
0 Hz (DC)	0 dB (통과)
10 Hz	~0 dB (통과)
20 Hz (차단 주파수)	~-6 dB (-3 dB 기준점)
30 Hz	~-25 dB
50 Hz (나이퀴스트)	< -40 dB

5. CMSIS-DSP 구현 상세

5.1 필요 구성 요소

```

#include <arm_math.h>                                /* CMSIS-DSP 헤더 */

#define FIR_TAP_COUNT    11U                          /* 탭 수 N */
#define ADC_CHANNEL_COUNT 3U                          /* 채널 수 */

arm_fir_instance_f32 g_fir[ADC_CHANNEL_COUNT];        /* 필터 인스턴스 */
float32_t             g_fir_state[ADC_CHANNEL_COUNT][ /* 상태 버퍼 */
                                                         [FIR_TAP_COUNT]; /* 크기 = N */

```

상태 버퍼 크기: 반드시 `FIR_TAP_COUNT` (= N) 이상의 요소를 가져야 함.

블록 처리 시에는 `N + blockSize - 1` 크기가 필요함.

5.2 초기화 (`arm_fir_init_f32`)

```
arm_status arm_fir_init_f32(
    arm_fir_instance_f32 *S, /* 필터 인스턴스 포인터 */
    uint16_t numTaps, /* 탭 수 N */
    const float32_t *pCoeffs, /* 계수 배열 (역순 불필요, 내부 처리) */
    float32_t *pState, /* 상태 버퍼 (N 요소, 0으로 초기화됨) */
    uint32_t blockSize /* 한 번에 처리할 샘플 수 */
);
```

본 프로젝트 호출 예시 (채널별 1샘플씩 처리):

```
arm_fir_init_f32(&g_fir[0], FIR_TAP_COUNT, g_fir_coefs, g_fir_state[0], 1U);
arm_fir_init_f32(&g_fir[1], FIR_TAP_COUNT, g_fir_coefs, g_fir_state[1], 1U);
arm_fir_init_f32(&g_fir[2], FIR_TAP_COUNT, g_fir_coefs, g_fir_state[2], 1U);
```

각 채널은 독립된 상태 버퍼를 사용하므로 채널 간 간섭 없음.

5.3 실행 (`arm_fir_f32`)

```
void arm_fir_f32(
    const arm_fir_instance_f32 *S, /* 초기화된 인스턴스 */
    const float32_t *pSrc, /* 입력 샘플 배열 */
    float32_t *pDst, /* 출력 샘플 배열 */
    uint32_t blockSize /* 처리할 샘플 수 */
);
```

본 프로젝트 호출 예시 (메인 루프에서 1샘플씩):

```
float32_t fir_input[3], fir_output[3];

fir_input[0] = (float32_t)frame.channel[0];
fir_input[1] = (float32_t)frame.channel[1];
fir_input[2] = (float32_t)frame.channel[2];

arm_fir_f32(&g_fir[0], &fir_input[0], &fir_output[0], 1U);
arm_fir_f32(&g_fir[1], &fir_input[1], &fir_output[1], 1U);
arm_fir_f32(&g_fir[2], &fir_input[2], &fir_output[2], 1U);
```

5.4 CMakeLists.txt에서 CMSIS-DSP 링크

```
target_link_libraries(${PROJECT_NAME}
    ...
    ${CMAKE_SOURCE_DIR}/Middlewares/Third_Party/ARM_CMSIS/Lib/GCC/libarm_cortexM4lf_math.a
)

target_compile_definitions(${PROJECT_NAME} PRIVATE
    ARM_MATH_CM4
    __FPU_PRESENT=1
)
```

6. 군지연(Group Delay)과 위상 특성

6.1 군지연 정의

$$\tau_g(\omega) = -\frac{d\phi(\omega)}{d\omega}$$

선형 위상 FIR 필터에서는 군지연이 주파수에 무관하게 일정함:

$$\tau_g = \frac{N-1}{2} \text{ samples} = \text{const}$$

6.2 비선형 위상과의 비교

특성	선형 위상 FIR	IIR (Butterworth 등)
군지연	일정 (모든 주파수 동일 지연)	주파수마다 다름
파형 왜곡	없음	있음 (위상 왜곡)
연산량	높음 (N번 곱셈)	낮음
안정성	항상 안정	설계 따라 불안정 가능

6.3 지연 보상 고려사항

서보 제어 루프에서 50 ms 지연이 허용 가능한지 확인 기준:

- 서보 응답 시간 (SG90): 100~200 ms
- 필터 지연: 50 ms
- 여유: $\approx 50\sim150$ ms $\rightarrow$ 허용 범위

더 빠른 응답이 필요하면 탭 수를 줄이거나 차단 주파수를 높여야 함.

7. FIR vs IIR 비교

항목	FIR	IIR
안정성	무조건 안정 (피드백 없음)	불안정 가능 (폴 위치 의존)
위상	선형 위상 가능	비선형 위상
연산량	$O(N)$ 곱셈/샘플	$O(M)$ 소수 곱셈으로 동등 성능
구현 복잡도	단순	상대적으로 복잡
계수 설계	직관적	아날로그 프로토타입 변환 필요
고정소수점	안정적	계수 민감도 높음
초기 상태	0으로 시작해 점진 수렴	동일

임베디드 저주파 신호 처리 $\rightarrow$ **FIR 권장**: 안정성, 선형 위상, CMSIS-DSP 최적화 지원

8. 필터 파라미터 변경 가이드

8.1 차단 주파수 변경

1. 아래 Python 스크립트를 실행하여 새 계수 생성
2. `g_fir_coeffs[]` 배열 교체
3. `FIR_CUTOFF_HZ` 매크로 값 수정

8.2 탭 수 변경

1. `FIR_TAP_COUNT` 매크로 변경
2. 계수 배열 크기 변경
3. 상태 버퍼 `g_fir_state[CH][FIR_TAP_COUNT]` 크기 자동 반영
4. 새 계수 생성 후 배열 교체

8.3 탭 수와 성능의 관계

탭 수	그룹 지연	천이 대역	저지 감쇠	MCU 부하
7	30 ms	넓음	낮음	낮음
11	50 ms	중간	중간	중간
21	100 ms	좁음	높음	높음
51	250 ms	매우 좁음	매우 높음	높음

8.4 샘플링 주파수 변경 시 주의사항

샘플링 주파수를 변경하면 정규화 차단 주파수가 바뀌므로 반드시 계수를 재설정해야 함.

예) $f_s = 200 \text{ Hz}$, $f_c = 20 \text{ Hz}$ 로 변경 시:

$$\omega_c = 20/200 = 0.1 \quad (0.2\text{에서 변경됨})$$

9. Python 계수 생성 스크립트

9.1 scipy.signal 사용 (권장)

```
#!/usr/bin/env python3
"""
FIR 저역통과 필터 계수 생성기
사용법: python3 generate_fir.py
"""

import numpy as np
from scipy.signal import firwin, freqz
import matplotlib.pyplot as plt

# — 파라미터 설정 —
FS      = 100.0    # 샘플링 주파수 [Hz]
FC      = 20.0     # 차단 주파수 [Hz]
N_TAPS  = 11       # 탭 수 (홀수 권장)
```



```

WINDOW      = 'hann'      # 창함수: 'hann', 'hamming', 'blackman', 'kaiser'
# -----

# 계수 생성 (firwin은 내부적으로 정규화 처리)
coeffs = firwin(N_TAPS, FC, window=WINDOW, fs=FS)

# C 배열 출력
print(f"/* {FS:.0f}Hz 샘플링 기준 {FC:.0f}Hz 저역통과 FIR 계수({N_TAPS}tap, {WINDOW} window) */")
print(f"static const float32_t g_fir_coeffs[{N_TAPS}U] = {{{")
for i, c in enumerate(coeffs):
    comma = "," if i < len(coeffs) - 1 else ""
    print(f"    {c:+.9f}{comma}    /* h[{i}] */")
print("};");

# DC 게인 확인
dc_gain = np.sum(coeffs)
print(f"\n/* DC 게인: {dc_gain:.6f} (이상값: 1.000000) */")
print(f"/* 그룹 지연: {(N_TAPS-1)//2} samples = {(N_TAPS-1)//2 * 1000/FS:.1f} ms */")

# 주파수 응답 플롯
w, h = freqz(coeffs, worN=2048, fs=FS)
plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.plot(w, 20 * np.log10(np.abs(h) + 1e-12))
plt.axvline(FC, color='r', linestyle='--', label=f'fc={FC}Hz')
plt.xlabel('Frequency [Hz]')
plt.ylabel('Magnitude [dB]')
plt.title(f'FIR LPF: {N_TAPS}-tap {WINDOW}, fc={FC}Hz, fs={FS}Hz')
plt.ylim(-80, 5)
plt.grid(True)
plt.legend()

plt.subplot(1, 2, 2)
plt.stem(range(N_TAPS), coeffs, markerfmt='C0o', basefmt='k-')
plt.xlabel('Tap index')
plt.ylabel('Coefficient value')
plt.title('Impulse Response (Coefficients)')
plt.grid(True)
plt.tight_layout()
plt.savefig('fir_response.png', dpi=150)
plt.show()

```

9.2 실행 환경 설정

```

# 가상환경 활성화 (프로젝트 .venv 사용)
source .venv/bin/activate

# 필요 패키지 설치 (최초 1회)
pip install numpy scipy matplotlib

# 스크립트 실행
python3 generate_fir.py

```

9.3 실행 결과 예시

```

/* 100Hz 샘플링 기준 20Hz 저역통과 FIR 계수(11tap, hann window) */

```

```
static const float32_t g_fir_coeffs[11U] = {
    -0.000000000f,    /* h[0] */
    -0.012641976f,    /* h[1] */
    -0.024692258f,    /* h[2] */
    +0.063505130f,    /* h[3] */
    +0.274797751f,    /* h[4] */
    +0.398062705f,    /* h[5] */
    +0.274797751f,    /* h[6] */
    +0.063505130f,    /* h[7] */
    -0.024692258f,    /* h[8] */
    -0.012641976f,    /* h[9] */
    -0.000000000f,    /* h[10] */
};

/* DC 게인: 1.000000 (이상값: 1.000000) */
/* 그룹 지연: 5 samples = 50.0 ms */
```

10. 자주 묻는 질문 / 트러블슈팅

Q1. 필터 출력이 초기에 0 근처 값이 나오는 이유는?

상태 버퍼가 0으로 초기화되어 있어 처음 $N - 1 = 10$ 개 샘플 동안은 과도 응답(Transient)이 발생함. 100 Hz 기준으로 **100 ms** 후 정상 동작함.

Q2. CMSIS-DSP 계수 순서는 뒤집혀야 하나요?

`arm_fir_init_f32`는 계수를 입력 순서 그대로 받아서 내부적으로 반전 처리함. `firwin` 출력을 그대로 사용하면 됨.

Q3. 탭 수를 늘렸는데 빌드 오류가 난다면?

```
/* FIR_TAP_COUNT 변경 후 상태 버퍼 자동 반영 확인 */
float32_t g_fir_state[ADC_CHANNEL_COUNT][FIR_TAP_COUNT]; /* OK */
```

상태 버퍼가 `FIR_TAP_COUNT`를 직접 참조하므로 매크로만 바꾸면 됨.

Q4. 필터 출력값이 원시 ADC 값보다 크게 튀는 경우

- 차단 주파수 근처에서 약간의 리플은 정상
- 큰 이상 출력이면 상태 버퍼 크기가 올바른지 확인
- `blockSize = 1`인데 `arm_fir_init_f32`에서 `blockSize` 파라미터를 다르게 줬는지 확인

Q5. 3채널 이상으로 확장하려면?

```
#define ADC_CHANNEL_COUNT 8U /* 채널 수 변경 */

arm_fir_instance_f32 g_fir[ADC_CHANNEL_COUNT];
float32_t g_fir_state[ADC_CHANNEL_COUNT][FIR_TAP_COUNT];

/* 초기화 반복문 */
for (int i = 0; i < ADC_CHANNEL_COUNT; i++) {
    arm_fir_init_f32(&g_fir[i], FIR_TAP_COUNT,
                    g_fir_coeffs, g_fir_state[i], 1U);
}
```

참고 자료

자료	설명
CMSIS-DSP API 문서	<code>arm_fir_f32</code> 공식 레퍼런스
scipy.signal.firwin	Python FIR 계수 생성
Proakis & Manolakis, <i>Digital Signal Processing</i>	FIR 설계 이론 교재
Smith, <i>The Scientist and Engineer's Guide to DSP</i>	무료 온라인 DSP 교재

최종 수정: 2025-04-26 | 작성자: JWLee